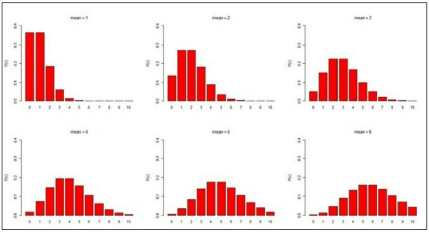

Figure 1: Poisson

probability of three failures in an observation of 10 instances:

```
>dbinom(3, size=10, prob=0.3);
>pbinom(3, size=10, prob=0.3);
```

The second statement computes the probability of three or less failures.

Gaussian distribution: This is the probability stating that an outcome will fall between two pre-calculated values. If the fraud data given above follows normal distribution, the following code computes the probability that the fraud cases in a month are less than 10:

```
>m <- mean(fraud_2013);
>s <- sd(fraud_2013);
> pnorm(9, mean=m, sd=s);
```

Graphical data analysis

Probabilistic plots: These are often needed to check and demonstrate fluctuating data values over some fixed range, like time. *dnorm*, *pnorm*, *qnorm* and *rnorm* are a few basic normal distributions that provide density, the distribution function, quantile function and random deviation, respectively, over the input mean and standard deviation. Let us generate some random number and check its probability density function. Such random generations are often used to train neural nets:

```
>ndata <- rnorm(100);
>ndata <- sort(ndata);
>hist(ndata,probability=TRUE);
>lines(density(ndata),col="blue");
>d <- dnorm(ndata);
>lines(ndata,d,col="red");
```

The Poisson distribution *(http://mathworld.wolfram.com/PoissonDistribution.html)* is used to get outcomes when the average successful outcome over a specified region is known. For the hypothetical insurance company, the average fraudulent claims related to the property and casualty sector in the first quarter of the year are given. Using the Poisson distribution, we can hypothesise the exact number of fraud claims. Figure 1 graphically plots the distribution for the exact number (0 to 10) of fraudulent claims with different means (1 to 6). From the figure, it is clear that as the rate increases, the high number of fraud claims also shift towards the right of the x axis.

```
par(mfrow=c(2,3))
avg <- 1:6
x <- 0:10
for(i in avg){
   pvar <- dpois(x, avg[i])
   barplot(pvar,names.arg=c("0", "1", "2", "3", "4",
"5", "6", "7", "8", "9", "10"),ylab=expression(P(x)),
ylim=c(0,.4), col="red")
   title(main = substitute(Rate == i,list(i=i)))
}  //Poisson.png
```

In the field of data analytics, we are often concerned with the coherency in data. One common way to ensure this is regression testing. It is a type of curve fitting (in case of multiple regressions) and checks the intercepts made at different instances of a fixed variable. Such techniques are widely used to check and select the convergence of data from the collections of random data. Regressions of a higher degree (two or more) produce a more accurate fitting, but if such a curve fitting produces a NaN error (not a number), just ignore those data points.

```
>attach(cars);
>lm(speed~dist);

>claim_2013 <- c(30,20,40,12,13,17,28,33,27,22,31,16);
>fraud_2013 <- c(8,4,11,3,1,2,11,17,12,10,8,7);
>plot(claim_2013,fraud_2013);
>poly2 <- lm(fraud_2013 ~ poly(claim_2013, 2, raw=TRUE));
>poly3 <- lm(fraud_2013 ~ poly(claim_2013, 3, raw=TRUE));
>summary(poly2);
>summary(poly3);
>plot(poly2);
>plot(poly3);
```

Interactive plots: One widely used package for interactive plotting is iplots. Use *install.packages("iplots")*; to install iplot and *library("iplots");* to load it in R. There are plenty of sample data sets in R that can be used for demonstration and learning purposes. You can issue the command *data();* at the R prompt to check the list of available datasets. To check the datasets available across